



Daniel Macharia <danielmacharia43@gmail.com>

YOHPAL LIVE RELEASE 1.0G — Final Evidence & GA Certification Review

Daniel Macharia <danielmacharia43@gmail.com>
To: <daniel@ics.ac.ke>

Fri, 10 Jul at 18:34

YOHPAL LIVE RELEASE 1.0G — Final Evidence & GA Certification Review

The latest verified implementation report still records **four Critical findings open, one Critical mitigated rather than fully closed, and zero of thirteen High findings fully closed**. Release 1.0G therefore begins as an **evidence gate**, not an automatic certification.

IMPLEMENTATION_STATUS_REPORT.pdf

Certification Objective

Independently verify that Release 1.0F has closed every remaining Critical finding and reduced the High-risk findings to a formally approved threshold before YohPal Live is exposed to general public users or unrestricted real-money activity.

Current Opening Decision

GA status: NO-GO pending Release 1.0F evidence.

The Release Review Board may change this only after reviewing committed code, automated test results, device proof, production observability evidence, security evidence, and the updated production-readiness audit.

1. Mandatory Evidence Package

Developers must submit one immutable evidence directory:

```
release-evidence/  
└─ yohpal-live-1.0g/  
    └─ 01-repository/  
    └─ 02-critical-closures/  
    └─ 03-high-closures/  
    └─ 04-security/  
    └─ 05-observability/  
    └─ 06-android/  
    └─ 07-ios/  
    └─ 08-tests/  
    └─ 09-performance/  
    └─ 10-readiness-review/  
    └─ 11-signatures/
```

Required evidence:

1. Git branch and full commit hash.
 2. Release candidate tag.
 3. Pull request and CI workflow links.
 4. Critical-finding closure matrix.
 5. High-finding closure matrix.
 6. Android regression report.
 7. iPhone regression report.
 8. Crashlytics release-build test event.
 9. Full-project flutter analyze.
 10. Full flutter test --coverage.
 11. Cloud Functions build and test output.
 12. Firestore rules test output.
 13. Navigation integrity output.
 14. Updated Production Readiness Review.
 15. Signed Release Review Board resolution.
-

2. Evidence Manifest Codebase

tool/release_certification/evidence_manifest.dart

```
import 'dart:convert';
import 'dart:io';

enum EvidenceStatus {
  pending,
  passed,
  failed,
  acceptedRisk,
}

final class EvidenceItem {
  const EvidenceItem({
    required this.id,
    required this.title,
    required this.severity,
    required this.status,
    required this.evidencePaths,
    this.notes,
  });

  final String id;
  final String title;
  final String severity;
  final EvidenceStatus status;
  final List<String> evidencePaths;
  final String? notes;

  Map<String, Object?> toJson() => {
    'id': id,
    'title': title,
    'severity': severity,
    'status': status.name,
    'evidencePaths': evidencePaths,
    'notes': notes,
  };

  factory EvidenceItem.fromJson(Map<String, dynamic> json) {
    return EvidenceItem(
      id: json['id'] as String,
      title: json['title'] as String,
      severity: json['severity'] as String,
      status: EvidenceStatus.values.byName(json['status'] as String),
      evidencePaths: List<String>.from(
        json['evidencePaths'] as List<dynamic>,
      ),
      notes: json['notes'] as String?,
    );
  }
}

final class ReleaseEvidenceManifest {
  const ReleaseEvidenceManifest({
    required this.release,
    required this.branch,
    required this.commitHash,
    required this.tag,
    required this.generatedAt,
    required this.items,
  });

  final String release;
  final String branch;
  final String commitHash;
  final String tag;
  final DateTime generatedAt;
  final List<EvidenceItem> items;

  bool get hasOpenCritical => items.any(
    (item) =>
      item.severity == 'critical' &&
      item.status != EvidenceStatus.passed,
  );

  bool get hasFailedHigh => items.any(
    (item) =>
      item.severity == 'high' &&
      item.status == EvidenceStatus.failed,
  );

  Map<String, Object?> toJson() => {
    'release': release,
    'branch': branch,
    'commitHash': commitHash,
    'tag': tag,
```

```

        'generatedAt': generatedAt.toUtc().toIso8601String(),
        'items': items.map((item) => item.toJson()).toList(),
    };
}

factory ReleaseEvidenceManifest.fromJson(
    Map<String, dynamic> json,
) {
    return ReleaseEvidenceManifest(
        release: json['release'] as String,
        branch: json['branch'] as String,
        commitHash: json['commitHash'] as String,
        tag: json['tag'] as String,
        generatedAt: DateTime.parse(json['generatedAt'] as String),
        items: (json['items'] as List<dynamic>)
            .map(
                (item) => EvidenceItem.fromJson(
                    item as Map<String, dynamic>,
                ),
            )
            .toList(),
    );
}

}

void main(List<String> args) {
    if (args.length != 1) {
        stderr.writeln(
            'Usage: dart run tool/release_certification/'
            'evidence_manifest.dart <manifest.json>',
        );
        exitCode = 64;
        return;
    }

    final file = File(args.first);

    if (!file.existsSync()) {
        stderr.writeln('Evidence manifest not found: ${file.path}');
        exitCode = 66;
        return;
    }

    final manifest = ReleaseEvidenceManifest.fromJson(
        jsonDecode(file.readAsStringSync()) as Map<String, dynamic>,
    );

    final missingFiles = <String>[];

    for (final item in manifest.items) {
        for (final evidencePath in item.evidencePaths) {
            if (!File(evidencePath).existsSync() &&
                !Directory(evidencePath).existsSync()) {
                missingFiles.add('${item.id}: $evidencePath');
            }
        }
    }

    if (missingFiles.isNotEmpty) {
        stderr.writeln('Missing evidence files:');
        for (final missing in missingFiles) {
            stderr.writeln(' - $missing');
        }
        exitCode = 1;
        return;
    }

    if (manifest.hasOpenCritical) {
        stderr.writeln(
            'Certification blocked: one or more Critical items '
            'have not passed.',
        );
        exitCode = 2;
        return;
    }

    if (manifest.hasFailedHigh) {
        stderr.writeln(
            'Certification blocked: one or more High items failed.',
        );
        exitCode = 3;
        return;
    }

    stdout.writeln(
        'Evidence manifest valid for ${manifest.release} '
        'at ${manifest.commitHash}.',
    );
}

```

```
}
```

3. Initial Manifest

release-evidence/yohpal-live-1.0g/manifest.json

```
{
  "release": "YohPal Live 1.0G",
  "branch": "flutter_app",
  "commitHash": "DEVELOPER_MUST_REPLACE",
  "tag": "yohpal-live-v1.0.0-rc1",
  "generatedAt": "2026-07-10T00:00:00Z",
  "items": [
    {
      "id": "C-02",
      "title": "Analytics route resolves without runtime crash",
      "severity": "critical",
      "status": "pending",
      "evidencePaths": [
        "release-evidence/yohpal-live-1.0g/02-critical-closures/analytics-route-test.txt",
        "release-evidence/yohpal-live-1.0g/02-critical-closures/analytics-route-device.png"
      ]
    },
    {
      "id": "C-03",
      "title": "Crashlytics and global error boundaries operational",
      "severity": "critical",
      "status": "pending",
      "evidencePaths": [
        "release-evidence/yohpal-live-1.0g/05-observability/crashlytics-test-event.png",
        "release-evidence/yohpal-live-1.0g/05-observability/global-error-handler.txt"
      ]
    },
    {
      "id": "C-04",
      "title": "Password reset works end to end",
      "severity": "critical",
      "status": "pending",
      "evidencePaths": [
        "release-evidence/yohpal-live-1.0g/02-critical-closures/password-reset-android.mp4",
        "release-evidence/yohpal-live-1.0g/02-critical-closures/password-reset-ios.mp4"
      ]
    },
    {
      "id": "C-05",
      "title": "Rewarded ads use real SDK and server-side verification",
      "severity": "critical",
      "status": "pending",
      "evidencePaths": [
        "release-evidence/yohpal-live-1.0g/04-security/rewarded-ad-ssv-test.txt"
      ]
    },
    {
      "id": "C-08",
      "title": "Production endpoints are environment configurable",
      "severity": "critical",
      "status": "pending",
      "evidencePaths": [
        "release-evidence/yohpal-live-1.0g/02-critical-closures/environment-config-test.txt"
      ]
    }
  ]
}
```

4. Automated GA Gate

scripts/run_yohpal_live_ga_gate.sh

```
#!/usr/bin/env bash
set -euo pipefail

APP_DIR="${APP_DIR:-apps/mobile_flutter}"
EVIDENCE_DIR="${EVIDENCE_DIR:-release-evidence/yohpal-live-1.0g}"

cd "$APP_DIR"

mkdir -p "../../$EVIDENCE_DIR/01-repository"
mkdir -p "../../$EVIDENCE_DIR/08-tests"
```

```

git branch --show-current \
| tee "../../$EVIDENCE_DIR/01-repository/branch.txt"

git rev-parse HEAD \
| tee "../../$EVIDENCE_DIR/01-repository/commit-hash.txt"

git status --short \
| tee "../../$EVIDENCE_DIR/01-repository/git-status.txt"

if [[ -s "../../$EVIDENCE_DIR/01-repository/git-status.txt" ]]; then
    echo "Working tree is not clean."
    exit 1
fi

flutter pub get

dart format \
  --output=none \
  --set-exit-if-changed \
  lib test tool \
| tee "../../$EVIDENCE_DIR/08-tests/dart-format.txt"

flutter analyze \
| tee "../../$EVIDENCE_DIR/08-tests/flutter-analyze.txt"

dart run tool/check_routes.dart \
| tee "../../$EVIDENCE_DIR/08-tests/route-integrity.txt"

flutter test --coverage \
| tee "../../$EVIDENCE_DIR/08-tests/flutter-test.txt"

if [[ -d functions ]]; then
    pushd functions >/dev/null

    npm ci
    npm run build \
    | tee "../../$EVIDENCE_DIR/08-tests/functions-build.txt"

    npm test \
    | tee "../../$EVIDENCE_DIR/08-tests/functions-test.txt"

    popd >/dev/null
fi

dart run tool/release_certification/evidence_manifest.dart \
  "../../$EVIDENCE_DIR/manifest.json"

echo "YohPal Live GA release gate passed."

```

5. CI Workflow

.github/workflows/yohpal_live_1_0g_certification.yml

```

name: YohPal Live 1.0G Certification

on:
  workflow_dispatch:
  pull_request:
    branches:
      - flutter_app
      - main
  push:
    tags:
      - "yohpal-live-v*"

permissions:
  contents: read

jobs:
  certify:
    runs-on: ubuntu-latest
    timeout-minutes: 40

    defaults:
      run:
        working-directory: apps/mobile_flutter

    steps:
      - name: Checkout exact commit
        uses: actions/checkout@v4
        with:
          fetch-depth: 0

```

- name: Install Flutter
uses: subosito/flutter-action@v2
with:
 channel: stable
 cache: true
- name: Install dependencies
run: flutter pub get
- name: Verify formatting
run: dart format --output=none --set-exit-if-changed lib test tool
- name: Analyze entire app
run: flutter analyze
- name: Route reachability gate
run: dart run tool/check_routes.dart
- name: Test Flutter application
run: flutter test --coverage
- name: Install Cloud Functions dependencies
working-directory: apps/mobile_flutter/functions
run: npm ci
- name: Build Cloud Functions
working-directory: apps/mobile_flutter/functions
run: npm run build
- name: Test Cloud Functions
working-directory: apps/mobile_flutter/functions
run: npm test
- name: Upload test evidence
if: always()
uses: actions/upload-artifact@v4
with:
 name: yohpal-live-1.0g-ci-evidence
 path: |
 apps/mobile_flutter/coverage/
 apps/mobile_flutter/test-results/
 apps/mobile_flutter/functions/coverage/

6. Critical-Finding Decision Matrix

Critical gate	Opening status	Evidence needed to pass
Sign-out reachable	Previously reported fixed	Android and iOS proof plus cross-account cleanup
Analytics route	Open	Automated route test and device proof
Crash reporting	Open	Release-build Crashlytics event
Password reset	Open	End-to-end Android and iOS proof
Rewarded-ad verification	Mitigated	Real ad SDK plus server-side verification
Gift validation	Previously reported fixed	Cloud Function and fraud test evidence
Wallet reconciliation	Previously reported fixed	Deep-link return and status proof
Environment abstraction	Open	Dev/staging/prod build proof

Firestore rules	Previously reported fixed	Emulator/security test output
-----------------	---------------------------	-------------------------------

No Critical gate may be accepted as a business risk for GA where it affects money, authentication, data integrity, fraud, or crash visibility.

7. High-Finding Threshold

Recommended GA rule:

```
enum ReleaseDecision {
    go,
    conditionalGo,
    noGo,
}

ReleaseDecision decideRelease({
    required int criticalOpen,
    required int criticalMitigated,
    required int highOpen,
    required bool androidPassed,
    required bool iosPassed,
    required bool observabilityPassed,
}) {
    if (criticalOpen > 0 || criticalMitigated > 0) {
        return ReleaseDecision.noGo;
    }

    if (!androidPassed || !iosPassed || !observabilityPassed) {
        return ReleaseDecision.noGo;
    }

    if (highOpen > 3) {
        return ReleaseDecision.conditionalGo;
    }

    return ReleaseDecision.go;
}
```

A **Conditional GO** should permit only a small percentage rollout behind feature flags, not full public GA.

8. Android Regression Evidence

Required Android journeys:

```
A01 Email signup
A02 Google Sign-In
A03 Login failure with friendly error
A04 Password reset
A05 Logout and second-account login
A06 Search navigation
A07 Live Discovery navigation
A08 Feed retry after forced network failure
A09 Pagination past 100 videos
A10 Mute/unmute
A11 Double-tap like
A12 Live stream network-drop recovery
A13 Wallet checkout return
A14 Gift purchase with insufficient balance
A15 Rewarded-ad verification
A16 Accessibility scan with TalkBack
A17 Crashlytics release-build test event
A18 Thirty-minute video playback
A19 Thirty-minute live-stream session
A20 Background/foreground restoration
```

Evidence must include device model, Android version, build number, timestamp, tester, result, screenshots or video, and logs.

9. iPhone Regression Evidence

Required iPhone journeys mirror Android, with additional checks for:

- Apple Sign-In.
- Associated domains and wallet return deep link.
- VoiceOver semantics.
- iOS background/foreground transitions.
- AVPlayer or LiveKit recovery.

- Physical-device Crashlytics test event.

The latest report specifically states that Google and Apple SSO were not yet exercised end to end in a real device environment, so source code alone cannot certify those paths.

IMPLEMENTATION_STATUS_REPORT.pdf

10. Production Observability Gate

The Release Review Board must see:

Crash-free users
Crash-free sessions
Fatal events
Non-fatal events
ANR rate
Startup failures
Feed load failures
Playback failures
Live reconnect attempts
Wallet return failures
Gift purchase failures
Authentication failures
Deep-link failures

Minimum observability evidence:

```
final class YohPalReleaseHealthThresholds {
    static const crashFreeSessions = 99.5;
    static const fatalCrashRateMaximum = 0.1;
    static const feedSuccessMinimum = 99.0;
    static const walletReturnSuccessMinimum = 99.0;
    static const liveReconnectSuccessMinimum = 95.0;
}
```

These are proposed thresholds and must be formally adopted by the Release Review Board before certification.

11. Release Review Board Resolution

docs/releases/YOHPALIVE_1_0G_RRB_RESOLUTION.md

YohPal Live 1.0G Release Review Board Resolution

Release Candidate

- Branch:
- Commit:
- Tag:
- Android build:
- iOS build:
- Review date:

Evidence Reviewed

- Critical closure register
- High closure register
- Flutter CI output
- Cloud Functions output
- Firestore rules tests
- Android regression report
- iPhone regression report
- Crashlytics proof
- Updated Production Readiness Review
- Security review
- Rollback proof

Gate Decisions

Gate	Decision	Evidence reference
Architecture		
Build		
Security		
Authentication		
Wallet and payments		
Feed and playback		
Live streaming		
Accessibility		
Observability		
Operations		

Final Decision

Select one:

- ☐ GO
- ☐ CONDITIONAL GO
- ☐ NO-GO

Approved Rollout

- ☐ Internal only
- ☐ 1%
- ☐ 5%
- ☐ 10%
- ☐ 25%
- ☐ 50%
- ☐ 100%

Conditions

- 1.
- 2.
- 3.

Rollback Triggers

- 1.
- 2.
- 3.

Signatures

Role	Name	Decision	Signature	Date
Flutter Lead				
Backend Lead				
Firebase Lead				
QA Lead				
Security Lead				
Product Owner				
Executive Sponsor				

12. Certification Outcome Rules

GO

May be issued only when:

- All Critical findings are fully closed.
- Android and iPhone validation pass.
- Crash reporting is proven in release builds.
- Security and financial controls pass.
- No unresolved P0/P1 incident exists.
- Rollback and emergency feature flags are verified.
- The refreshed readiness review contains no GA blocker.

CONDITIONAL GO

Permits a controlled rollout only when:

- All Critical findings are closed.
- Remaining High findings have named owners and deadlines.
- Their risks are bounded by feature flags or restricted rollout.
- Mission Control monitoring is active.
- Executive Sponsor approves the limitation.

NO-GO

Required when:

- Any Critical finding remains open or merely mitigated.
- Android or iPhone regression fails.
- Crash visibility is not proven.
- Financial, authentication, security, or data-integrity controls are uncertain.
- Evidence is missing, stale, unsigned, or not tied to the candidate commit.

Current Independent Decision

NO-GO for General Availability.

This is not a judgment on the proposed Release 1.0F code pack. It reflects the fact that committed implementation evidence, device regression results, Crashlytics proof, refreshed audit results, and board signatures have not yet been submitted. The most recent evidence still records material Critical and High gaps.

IMPLEMENTATION_STATUS_REPORT.pdf

Next Smart Move

Complete Release 1.0F against one frozen release-candidate commit, generate the full 1.0G evidence directory using the certification scripts above, and reconvene the Release Review Board only after all Critical gates show **Passed** rather than **Pending** or **Mitigated**.